

Machine Learning for Network Attacks Classification and Statistical Evaluation of Adversarial Learning Methodologies for Synthetic Data Generation

Iakovos-Christos Zarkadis
University of Piraeus
Athens, Greece
iakovos.zarkadis@gmail.com

Christos Douligeris
Dept. of Informatics
University of Piraeus
Piraeus, Greece
cdoulig@unipi.gr

Abstract—Supervised detection of network attacks has always been a critical part of network intrusion detection systems (NIDS). Nowadays, in a pivotal time for artificial intelligence (AI), with even more sophisticated attacks that utilize advanced techniques, such as generative artificial intelligence (GenAI) and reinforcement learning, it has become a vital component if we wish to protect our personal data, which are scattered across the web. In this paper, we address two tasks, in the first unified multi-modal NIDS dataset, which incorporates flow-level data, packet payload information and temporal contextual features, from the reprocessed CIC-IDS-2017, CIC-IoT-2023, UNSW-NB15 and CIC-DDoS-2019, with the same feature space. In the first task we use machine learning (ML) algorithms, with stratified cross validation, in order to prevent network attacks, with stability and reliability. In the second task we use adversarial learning algorithms to generate synthetic data, compare them with the real ones and evaluate their fidelity, utility and privacy using the SDV framework, f -divergences, distinguishability and non-parametric statistical tests. The findings provide stable ML models for intrusion detection and generative models with high fidelity and utility, by combining the Synthetic Data Vault framework, the TRTS and TSTR tests, with non-parametric statistical tests and f -divergence measures.

Index Terms—Machine learning (ML), artificial intelligence (AI), generative artificial intelligence (GenAI), network intrusion detection systems (NIDS), Unified Multimodal Network Intrusion Detection System (UMNIDS), generative adversarial networks (GANs).

I. INTRODUCTION

In the era of AI, detecting the type of the attack becomes even more difficult. Machine learning algorithms have been implemented in many research papers and application areas [10] showing efficiency in tackling attacks across a variety of well-known intrusion detection system (IDS) datasets [38], [24], [29], [30], [32]. Nevertheless, it is necessary to not simply focus on achieving high results in classification metrics, but offering reliability and stability to an intrusion detection system.

In the last few years adversarial attacks have gained popularity due to their sophisticated attacks, that utilize adversarial machine learning algorithms to inject perturbations in the data,

in order to make a strong classifier unable to classify certain attacks correctly, thus misleading the IDS.

Many types of algorithms, from linear models and SVM to neural networks and reinforcement learning have been proposed in [5]. Commonly used algorithms in IoT tabular data are logistic regression as a baseline for multi-class classification as well as more advanced models like deep neural networks and ensemble models [9]. More detailed studies have examined cross-dataset validation with LDA, Xgboost and decision trees [24]. Different frameworks have been widely used for data generation purposes like Synthcity [40] and SDV [26], [8] offering high quality generators and evaluating metrics. The attention of researchers has recently been directed in many conditional architectures for synthetic network data generation for GANs and VAEs [20], [21], [22], [25], [27], [31], as well as in the emerging f -GANs and diffusion models [6], [12], [39] with the LLMs being top-tier models [37]. Furthermore, the specific non-parametric statistical tests, for means, covariance matrices and multivariate distribution comparison, have been used in some papers considering network data [14], [17], [19], but may have never been combined to offer a complete view. Our focus, in this paper is not how to perform such attacks but on how the generation process evolves and test different adversarial algorithms, diffusion forests and LLM to create synthetic data [27], [31] and to evaluate which are the most efficient ones, in terms of fidelity, utility and privacy. In this paper we use TRTS, TSTR tests, supervised distinguishability tests, f -divergences, privacy measurement [11] and combine for the first time the SDV framework [34], with robust non-parametric statistical tests for comparing multivariate means, covariance matrices and the feature's joint multivariate distributions [14], [17], [19].

This work is organized in two sections. In the first we present the classification of the attacks, where we discuss data collection, pre-processing steps and the ML models used for intrusion detection. In the second section we show the generation process of synthetic data, including a short review of the mathematical literature, the training of adversarial, diffusion

and LLM algorithms and finally the evaluation procedure.

II. MACHINE LEARNING-BASED NIDS METHODOLOGY

A. Data Collection

For all the results that will be presented in the next sections we used the under-sampled version of the UMNIDS dataset, a newly created dataset, which was created by processing raw pcap files from CIC-IDS 2017, CIC-IOT 2023, UNSW-NB15 and CIC-DDoS 2017 [34]. UMNIDS includes flow-level and payload-based data with features that capture historical and temporal patterns in network traffic. This dataset includes around 170,000 observations, 88 features and 1 target variable, which implies if a record is normal or if it is an attack in the network along with the type of the attack. Some that are included are Worms, Bots, DDoS, Infiltration and SQL-Injection. The target variable includes a total of 44 unique values, 43 of which specify the kind of attack and the one remaining the non-malicious records. One of the main issues with this dataset is the lack of records for some attacks, like Heartbleed and Infiltrations.

B. Data Pre-Processing

The pre-processing procedure started by omitting the duplicate values, then encoding the label variable, testing different methods like binning for certain numeric variables with a small number of unique values. We performed an in-depth statistical analysis that included the discovering of linear and non-linear relationships, hypothesis testing for existence of normality through QQ-Plots and goodness of fit tests like Kolmogorov-Smirnov and Anderson Darling, as well as regression analysis for some numeric features. We also examined the existence of outliers with IQR, Z-score, DBSCAN and Local-Outlier Factor and due the great number of outliers we preferred to keep them and use as a scaling method the Robust scaler, after we performed an 80/20 train/test stratified split, with respect to the attack type variable.

Furthermore a great deal of feature engineering techniques were used to increase the efficiency of the ML algorithms, separately for the training and testing data. To tackle the problem of the unbalanced dataset, we investigated different over-sampling and under-sampling techniques only for the training set to avoid data leakage, like synthetic minority oversampling (SMOTE), edited nearest neighbor (ENN) and adaptive synthetic sampling (ADASYN) that lead us to select the over-sampling approach to reduce the impact of information loss. Finally for the feature selection procedure we tested multiple selection techniques like recursive feature elimination, stepwise (forward-backward) elimination and boruta.

C. Attacks Classification

For the classification of the tabular data, we selected baseline and top-tier models from different families. The two baseline models that we used for the multi-class classification task are Logistic Regression and the stochastic gradient descent-based SVM. The tree-based model that we utilized is a Decision Tree. The ensemble models are LightGBM, XGBoost,

XGBoost with Random Forest, Random Forest, Extra Trees, Gradient Boosting, and Histogram-Based Gradient Boosting. Finally we also chose to use the AutoGluon, an ensemble of neural networks, extra trees, XGBoost, LightGBM and linear models. Several hyperparameter optimization techniques were tested, such as grid-search, halving-grid-search, and Bayesian optimization, along with stratified 10-fold cross validation. After the selection of the hyperparameters, each of the pre-mentioned models has been trained with stratified 10-fold cross validation to observe their behavior without probability calibration. After that, we performed stratified 10-fold cross validation with probability calibration, while holding a 4% value range threshold for considering a model stable per metric, and obtained the results that follow for the training and testing set.

The metrics with which we evaluated the performance of each algorithm are F1, Precision, Recall, and Roc-Auc. From the results in Tab. I and Tab. II we observe the high performance of ensemble and tree-based models, and more specifically of XGBoost, LightGBM, AutoGluon, XGBoost-RF, Random Forest, Extra Trees, and Decision Tree, while the baseline models, SVM and Logistic Regression fail. To highlight the impressive performance of the non-calibrated XGBoost, in terms of both stability and efficiency, we present the cross validation results for precision and recall in Fig. 1.

TABLE I
TRAIN/TEST PRECISION AND RECALL RESULTS (%)

<i>Model</i>	<i>Train/Test-Precision</i>	<i>Train/Test-Recall</i>
LightGBM	97.74/96.36	97.69/96.25
AutoGluon	92.34/93.53	91.32/92.30
XGBoost	98.67/96.40	98.64/96.29
XGBoost-RF	96.04/95.09	95.99/94.74
Random Forest	96.64/95.88	96.56/95.63
HB-Gradient Boosting	94.74/94.74	93.91/93.91
Gradient Boosting	71.82/75.17	71.03/73.47
Extra Trees	97.05/96.30	96.91/96.09
Decision Tree	96.92/95.28	96.85/95.03
SVM	7.08/5.57	0.32/3.41
Logistic Regression	8.16/6.40	6.47/6.82

TABLE II
TRAIN/TEST F1 AND ROC-AUC RESULTS (%)

<i>Model</i>	<i>Train/Test-F1</i>	<i>Train/Test-Roc-Auc</i>
LightGBM	97.69/96.25	98.81/98.08
AutoGluon	91.12/92.31	95.56/9606
XGBoost	98.65/96.33	99.30/98.11
XGBoost-RF	95.99/94.86	98.95/97.32
Random Forest	95.56/95.49	96.56/95.63
HB-Gradient Boosting	94.66/94.55	97.28/97.18
Gradient Boosting	69.14/72.78	85.18/86.64
Extra Trees	96.92/96.15	98.42/98.00
Decision Tree	96.85/95.11	98.37/97.46
SVM	1.29/1.34	50.45/50.50
Logistic Regression	5.05/4.84	52.15/52.16

III. GENERATION OF SYNTHETIC DATA

The generation of synthetic data has gained popularity nowadays, especially with the hype of GenAI. It is very interesting

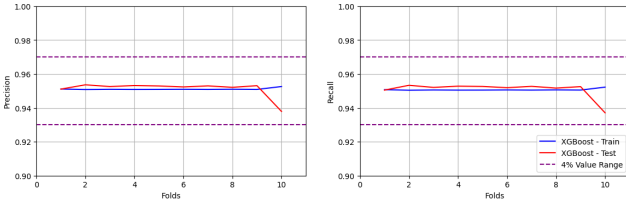


Fig. 1. XGBoost Precision-Recall Stratified 10-Fold Cross Validation.

how the generation of fake data can trick a machine learning based intrusion detection system, but even more interesting how we can use such generation techniques to construct synthetic datasets, that share statistical properties with the real ones.

A. Short Review of the Architecture of Generative Adversarial Networks and their Loss Functions

A generative adversarial network architecture is consisted of two algorithms competing each other in a minmax game [18]. The first algorithm is called Generator, whose role is to generate synthetic data identical to the real ones and is commonly noted as G . The second one is called Discriminator, whose role is to distinguish between real and synthetic data and is commonly noted as D . For the most simple architectures D and G are neural networks. Let a training data-set of dimensions $n \times p$, where n is the number of observations and p is the number of features, while each record $\mathbf{x}_i \in \mathcal{X}^p$, $i = 1, 2, \dots, n$. The generator takes as input random noise, let $\mathbf{z} \in R^k$, from a k -dimensional distribution and maps it to a p -dimensional space, thus generating a new fake data-point, let $x_{fake}^{(k)} = G(\mathbf{z}) \in \mathcal{X}^p$, $k = 1, 2, \dots, m$, where m is the number of newly generated data-points. Now let $D(\mathbf{x}_i) \in [0, 1]$ be the probability of the i -th data-point to be real in this binary classification task. To be able to generate new synthetic data, the generator must learn how to create data identical to the real ones. This happens through the minmax game, which is a process of optimizing the parameters of each algorithm, i.e. the weights of each neural network. Thus, given the parameters θ_D, θ_G of the discriminator and the generator respectively we can notate them as $D(\mathbf{x}_i; \theta_D)$ and $G(\mathbf{z}; \theta_G)$ respectively [18]. The stepping-stone for optimizing the procedure is the binary cross-entropy risk function (BCE), a special occasion of the categorical cross-entropy risk function (CCE), when we address a task with two probable outcomes.

Given the discrete probability mass distributions $p(\cdot)$ and $q(\cdot)$, for a total of n data-points and C outcomes, the formulation of the risk functions of categorical cross-entropy and binary cross-entropy are formulated as seen in “(1)” and “(2)” [35], [36]:

$$R_{CCE}(p(x), q(x)) = \frac{1}{n} \sum_{i=1}^n \sum_{j=0}^{C-1} p(x_{ij}) \log_2(q(y_{ij})) \quad (1)$$

$$\begin{aligned} R_{BCE}(p(x), q(x)) &= \frac{1}{n} \sum_{i=1}^n p(x_{i0}) \log_2 q(y_{i0}) + \\ &+ \frac{1}{n} \sum_{i=1}^n p(x_{i1}) \log_2 q(y_{i1}) = \frac{1}{n} \sum_{i=1}^n p(x_{i0}) \log_2 q(y_{i0}) + \\ &+ \frac{1}{n} \sum_{i=1}^n (1 - p(x_{i0})) \log_2 (1 - q(y_{i0})) \end{aligned} \quad (2)$$

The last formulation of the binary cross-entropy is the most commonly used and can be explained since $p(y_0) + p(y_1) = 1$, where $S_Y = \{y_0, y_1\}$, where $p(\cdot)$ is a discrete probability mass distribution, with two outcomes, such that $S_Y = \{y_0, y_1\}$. From probability theory we have the following definition for the expected value of a transformation $g(X)$ of a discrete random variable X , with $f_X(x) \in [0, 1]$ as the probability mass function and $S_X = \{x_1, x_2\}$ [3]:

$$\begin{aligned} E_{f_X(x)}\{g(X)\} &= \sum_{x \in S_X} g(x) f_X(x) = \\ &= g(x_1) f_X(x_1) + g(x_2) f_X(x_2) = \\ &= g(x_1) f_X(x_1) + g(x_2) (1 - f_X(x_1)) \end{aligned} \quad (3)$$

Moving on from the theoretical background to the computational approach, given $\mathbf{X} \sim p(\mathbf{x})$, $\mathbf{Z} \sim q(\mathbf{z})$, where $p(\mathbf{x}), q(\mathbf{z})$ are two probability distributions, such that $p(\mathbf{x}) = 1.0$ if $\mathbf{x}$ is real, $p(\mathbf{x}) = 0.0$ if $\mathbf{x}$ is fake, one formulation of the risk function used in the Vanilla-GAN, from keras and pytorch library can be seen in “(4)” as follows [36]:

$$\begin{aligned} R_{Vanilla-GAN}(D(\mathbf{x}; \theta_D), G(\mathbf{z}; \theta_G)) &= \\ &= \frac{1}{n} \sum_{i=1}^n E_{p(\mathbf{x})} \left\{ \ln(D(\mathbf{x}_i; \theta_D)) \right\} + \\ &+ \frac{1}{n} \sum_{j=1}^n E_{q(\mathbf{z})} \left\{ \ln(1 - D(G(\mathbf{z}_j; \theta_G); \theta_D)) \right\} = \\ &= \frac{1}{n} \sum_{i=1}^n \left\{ p(\mathbf{x}_i) \ln(D(\mathbf{x}_i; \theta_D)) + \right. \\ &\quad \left. + (1 - p(\mathbf{x}_i)) \ln(1 - D(\mathbf{x}_i; \theta_D)) \right\} + \\ &+ \frac{1}{n} \sum_{j=1}^n \left\{ q(\mathbf{z}_j) \ln(1 - D(G(\mathbf{z}_j; \theta_G); \theta_D)) + \right. \\ &\quad \left. + (1 - q(\mathbf{z}_j)) \ln(D(G(\mathbf{z}_j; \theta_G); \theta_D)) \right\} = \\ &= \frac{1}{n} \sum_{i=1}^n \ln(D(\mathbf{x}_i; \theta_D)) + \\ &+ \frac{1}{n} \sum_{j=1}^n \ln(1 - D(G(\mathbf{z}_j; \theta_G); \theta_D)) \end{aligned} \quad (4)$$

The minmax procedure optimizes the following risk function [7], [13], [36] and more specifically it maximizes with respect to the discriminator D and minimizes with respect to the generator G as:

$$\min_G \max_D R_{Vanilla-GAN}(D(\mathbf{x}; \theta_D), G(\mathbf{z}; \theta_G)) \quad (5)$$

For the conditional architecture, we add a condition in the data, the target variable, which is categorical, after it is One-Hot encoded. In this architecture, the i -th input is not $\mathbf{x}_i$ as before, but has the form $(\mathbf{x}_i, \mathbf{y}_i)$, where $\mathbf{y}_i$ can be the encoded label for the i -th observation $\mathbf{x}_i$. In the basic architecture of CGAN, the minimax procedure has the following form, as seen in [36]:

$$\min_G \max_D R_{CGAN}(D((\mathbf{x}, \mathbf{y}); \theta_D), G((\mathbf{z}, \mathbf{y}); \theta_G)) \quad (6)$$

Moving to the f -GAN architectures, also known as variational divergence minimization [33], the objective function is based on the idea of the convex conjugate function $f^*(t) = \sup_{u \in \text{dom}_f} \{ut - f(u)\}$, the Fenchel conjugate, of a convex, lower-semicontinuous function $f(u) = \sup_{t \in \text{dom}_{f^*}} \{tu - f^*(t)\}$. The minimax game occurs between the generator and the discriminator, or otherwise the variational function and are noted as $D(\mathbf{x}; \theta_D)$ (or $T_{\theta_D}(\mathbf{x}) = g_f(V_{\theta_D}(\mathbf{x}))$) and $G(\mathbf{z}; \theta_G)$ respectively. Given an activation function $g_f(\cdot)$, related to the conjugate $f^*(\cdot)$, for $V_{\theta_D}(\mathbf{x})$, the objective function, of the minmax process is formulated as seen is “(7)”:

$$F = \left\{ E_{p(\mathbf{x})}(D(\mathbf{x}; \theta_D)) - E_{q(\mathbf{x})}(f^*(G(\mathbf{z}; \theta_G))) \right\} \quad (7)$$

The generator functions $f(u)$, along with their conjugate $f^*(t)$, for each f -divergence [33], which we used can be seen in Tab. III:

TABLE III
f-GAN GENERATOR FUNCTIONS AND THEIR CONJUGATES

Divergence	$f(u)$	$f^*(t)$
Kullback-Leibler	$u \log(u)$	$\exp\{t - 1\}$
Squared-Hellinger	$(\sqrt{u} - 1)^2$	$\frac{t}{1-t}$

B. Training Adversarial Models

For the generation of new synthetic data, we decided to select a wide range of different adversarial architectures. We chose Vanilla-GAN, Wasserstein-GAN (WGAN) [23], Wasserstein-GAN with gradient penalty (WGAN-GP) [23], f -GANs with Kullback-Leibler (KLD-GAN) and Squared-Hellinger divergence (H2D-GAN) [33], the basic architecture of Conditional-GAN for tabular data (CTGAN-1) [12], a Conditional-GAN equipped with Bayesian-Gaussian Mixture (CTGAN-2) [21], a Variational Auto-Encoder (TVAE) [26], an XGBoost-based Diffusion Forest (XGB-DF) [12], the DistilGPT2 [12] and the PATE-CTGAN, a specialized privacy-preserving conditional generative adversarial network, which utilizes the private aggregation of teacher ensembles architecture (PATE) [15].

For the training of the adversarial networks we tested a few neural network architectures, we consider a total number of 20 epochs for each of the above-mentioned algorithms. From the minimax procedure, for illustration purposes we can see below in Fig. 2 the discriminator’s and generator’s loss values for a few batches, for the Vanilla-GAN and WGAN, from which we observe the failure of Vanilla-GAN as the discriminator’s

loss converges to zero and the stable training WGAN, along with indications of convergence:

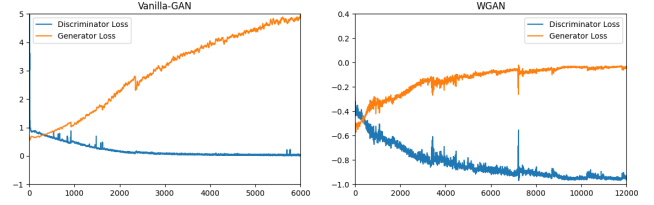


Fig. 2. GAN and WGAN Loss Progress.

Furthermore we highlight the loss progress of TVAE, whose results are the highest, compared to the rest non-conditional architectures. Although it fails generating equal number of observations per network attack. It’s performance could be further optimized with the use of a conditional architecture and a mixture approach:

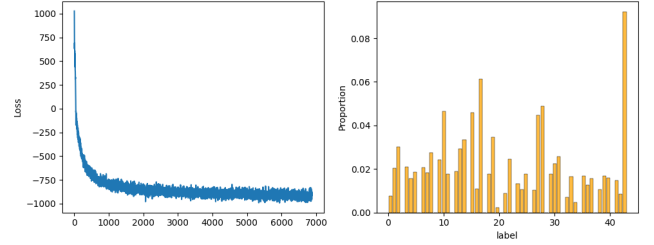


Fig. 3. TVAE Results

C. Evaluation

For evaluating the quality of the new synthetically generated data we chose the Synthetic Data Vault framework (SDV) [26]. From the quality report, we investigated the statistical fidelity, measuring how well we have captured the marginal distribution of each feature using the Kolmogorov-Smirnov Complement and how well the correlations have been preserved using the Correlation Similarity, thus leading us to a final score which is the average of the two pre-mentioned scores. We also run the diagnostic report, in order to verify if the structural integrity and data validity have been maintained with the Table Structure score, which investigates if data-types per feature between the real and the synthetic are the same and the Boundary Adherence, which checks if the minimum-maximum boundaries per feature are respected. As before with the quality report we have a final score for the diagnostic report, which is the average of the Table Structure score and the Boundary Adherence score. For the computation of the above-mentioned metrics we used only the training set. We consider the 65% and 95% thresholds for the averaged Quality and Diagnostic report scores, to select a few algorithms to proceed to further evaluation. The averaged scores can be seen in Tab. IV, where we highlight the performance of CTGAN-2 [21], the diffusion forest and the LLM [12].

After the selection of the models, we move forward with the Distinguishability Test, in order to further investigate if

TABLE IV
SDV OVERALL SCORES

<i>Model</i>	<i>Quality Report (%)</i>	<i>Diagnostic Report (%)</i>
Vanilla-GAN	60.43	78.37
WGAN	57.07	81.24
WGAN-GP	63.55	78.72
KLD-GAN	63.55	78.72
H2D-GAN	49.85	80.22
CTGAN-1	75.82	83.79
CTGAN-2	98.91	100.00
TVAE	83.15	86.52
XGB-DF	99.93	100.00
DistilGPT2	99.94	100.00
PATE-CTGAN	65.00	95.17

the synthetic data mimic the statistical properties of the real data. We consider two binary classifiers, a Random-Forest and the Logistic Regression as a baseline. From the baseline model results in Tab. VI we observe that it decides randomly whether it's a real or a fake data-point since the Roc-Auc score is almost 50% and the F1 score is 0%, for the first three models. For the differentially-privacy enhanced model we see that the baseline binary classifier, can't distinguish easily but tends to realize real from fake. As seen in Tab. V we see a good F1 score, fluctuating around 52% for all models, but from the Roc-Auc we have indications that the Random Forest discriminator struggles, but slightly favors the real data, since the scores are close to 64%, which indicates that there is space for improvement. The only generative model, whose synthetic data can be easily distinguished by the random forest, is the PATE-CTGAN.

TABLE V
RANDOM FOREST - DISTINGUISHABILITY TEST RESULTS (%)

<i>Model</i>	<i>Test-F1</i>	<i>Test-Roc-Auc</i>
CTGAN-2	54.24	64.24
XGB-DF	53.39	65.07
DistilGPT2	52.80	63.81
PATE-CTGAN	100.00	100.00

TABLE VI
LOGISTIC REGRESSION - DISTINGUISHABILITY TEST RESULTS (%)

<i>Model</i>	<i>Test-F1</i>	<i>Test-Roc-Auc</i>
CTGAN-2	0.00	50.85
XGB-DF	0.13	52.80
DistilGPT2	0.20	53.70
PATE-CTGAN	14.65	63.18

To further investigate the diversity and utility of the new generated data-sets and the scenario of mode collapse, we the perform Train-Real-Test-Synthetic (TRTS) and Train-Synthetic-Test-Real (TSTR) and compare the results with the Train-Real-Test-Real (TRTR) tests. For this comparison we selected the Random Forest classifier and as evaluation metrics the precision and recall score. In Tab. VII we can find the precision and recall results for the synthetic data, which acted as a test set. From these scores we observe that they match the precision and recall scores for the original test data, apart from

those of the PATE-CTGAN. From Tab. VIII we observe that the precision and recall scores of a newly trained Random-Forest, on the synthetic data, are fluctuating in the same levels of the scores seen in TRTR and TRTS. Though from PATE-CTGAN we have clear indications of poor data quality and lack of realism, for machine learning based tasks. All these results indicate high utility-quality data, with diversity and absence of mode collapse in CTGAN-2, XGB-DF and DistilGPT2.

TABLE VII
TRTS RESULTS (%)

<i>Model</i>	<i>Precision</i>	<i>Recall</i>
CTGAN-2	96.59	96.51
XGB-DF	96.67	96.59
DistilGPT2	96.67	96.59
PATE-CTGAN	00.54	0.02

TABLE VIII
TSTR RESULTS (%)

<i>Model</i>	<i>Train/Test Precision</i>	<i>Train/Test Recall</i>
CTGAN-2	96.60/95.65	96.51/95.38
XGB-DF	96.76/95.56	96.67/95.26
DistilGPT2	96.77/95.56	96.67/95.26
PATE-CTGAN	75.22/00.18	11.27/3.60

We also investigated how well each generative model has captured the distribution of the data, by comparing them with a few divergence measures [18], as seen below in Tab. IX. Almost all models achieved, at a significant level, to generate sufficient samples from each attack category, thus capturing the underlying univariate distribution of the target variable.

TABLE IX
DIVERGENCE-BASED EVALUATION METRICS

<i>Divergence</i>	<i>Jensen-Shannon</i>	<i>Hellinger</i>	<i>Wasserstein</i>
CTGAN-2	0.000967	0.031219	0.129161
XGB-DF	3.541914e-10	1.881997e-5	0.000106
DistilGPT2	3.541914e-10	1.881997e-5	0.000106
PATE-CTGAN	0.004467	0.066443	0.729779

For illustration purposes we show below in Fig. 4 and Fig. 5 the relative frequency per attack that was generated from CTGAN-2, the PATE-CTGAN and DistilGPT2, compared to the real proportions from the training set.

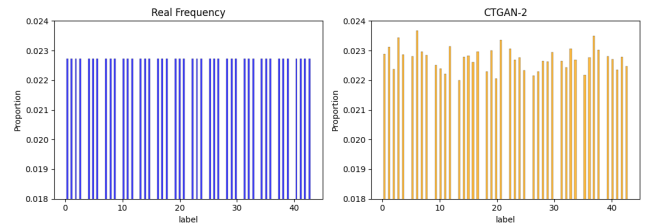


Fig. 4.

We also performed statistical hypothesis tests in order to compare the multivariate means and the covariance matrices

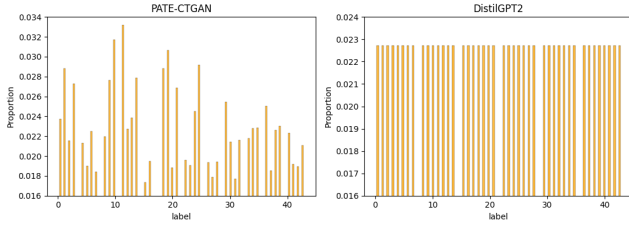


Fig. 5.

between the real and the synthetic data. We utilized the Regularized Hotelling's T^2 test [19] and the Frobenius Norm Covariance test [14], [16], for the multivariate means and covariance test respectively, each of which was computed for a total of 500 permutations, to conclude to the p -values seen in Tab. X. We consider a significance level of $\alpha = 5\%$, for both statistical tests seen in "(8)" and "(9)". Regarding the equality of means, from all the below p -values, as seen in Tab. X, we have evidence to reject the alternative hypothesis H_1 , at a significance level $\alpha = 0.05$, since p -value $> \alpha$, which leads us into the conclusion that the difference between the multivariate means of the real and synthetic data are not statistically significant. From the p -values of the Frobenius test, as seen in Tab. XI, we have evidence to reject the alternative hypothesis, at a significance level $\alpha = 0.05$, since p -value $> \alpha$, in all tests, except the one considering the CTGAN-2 model, from which we have clear indications to reject the null hypothesis H_0 , at a significance level $\alpha = 0.05$, since p -value $< \alpha$, meaning the difference between the covariance matrices of the real and synthetic data is statistically different at a significance level of $\alpha = 0.05$.

$$H_0 : \mu_1 = \mu_2 \text{ against } H_1 : \mu_1 \neq \mu_2 \quad (8)$$

$$H_0 : \Sigma_1 = \Sigma_2 \text{ against } H_1 : \Sigma_1 \neq \Sigma_2 \quad (9)$$

TABLE X
REGULARIZED HOTELLING'S T^2

Model	p -values
CTGAN-2	0.99
XGB-DF	1.00
DistilGPT2	1.00
PATE-CTGAN	0.99

TABLE XI
FROBENIUS NORM COVARIANCE TEST RESULTS

Model	p -values
CTGAN-2	0.0019
XGB-DF	0.9381
DistilGPT2	0.9261
PATE-CTGAN	0.14

We applied a two-sided non-parametric statistical test, Maximum Mean Discrepancy (MMD) [17], [4] to measure the distance between the joint distribution of the features of

the real and synthetic datasets, with 500 permutations in randomized samples of the original real and synthetic datasets, which computes the mean embeddings in a Reproducing Kernel Hilbert Space (RKHS). We consider a significance level of $\alpha = 5\%$ and their p -values can be seen in Tab. XII, for each generative model. From the table, we observe that we have no evidence to reject the null hypothesis ($H_0 : P = Q$) at a significance level of $\alpha = 5\%$, apart from the PATE-CTGAN model, which eventually failed to capture the multivariate distribution, mostly because of the differential privacy constraints. Thus, we can't assume that the difference between their distributions is statistically different for the CTGAN-2, the diffusion forest and the LLM.

TABLE XII
MMD TEST RESULTS

Model	p -values
CTGAN-2	0.0829
XGB-DF	1.0000
DistilGPT2	1.0000
PATE-CTGAN	0.0000

For the scenario of data leakage we use a privacy evaluation score for the generated data by each of the selected models [11], [28], to check if they are simply copying the existing records instead of synthesizing new ones, but also weight the trade-off between privacy and utility-quality. We utilized the Nearest Neighbor Distance Record score (NNDR). From Tab. XIII we observe the NNDR scores between the train and test sets compared to the synthetic datasets, to measure overfitting, while considering a threshold of 4% for the score difference between the train and the test sets. From the below scores, we have good NNDR scores, showing standard to high levels of privacy, especially for the PATE-CTGAN, but we have indications of overfitting for the other models, which don't have a privacy-preserving architecture.

TABLE XIII
NNDR RESULTS

Model	Train-NNDR	Test-NNDR
CTGAN-2	0.6136	0.7171
XGB-DF	0.6122	0.7194
DistilGPT2	0.6140	0.7193
PATE-CTGAN	0.9634	0.9731

IV. CONCLUSIONS

In this paper we clarified how efficient, in terms of speed, accuracy and stability, the tree and ensemble models are in classifying network attacks compared to other baseline models, when the network data are processed in tabular format. The conditional architectures, when combined with mixture models have outstanding performance in capturing the probability distribution function of the conditional variable, contrast to basic conditional architectures. We also highlight the diffusion and LLM models for their performance in the distinguishability tests and for capturing the underlying distributions

and the quality-privacy tradeoff. The use of robust and non-parametric statistical tests for analyzing the multivariate nature of the synthetic data, generated either from non-differential or differential architectures, along with the divergence measures give us a solid perspective of the univariate distribution of the attacks in the network and the joint distribution of the features. Future work includes cross-dataset validation with other well known IDS datasets, using adversarial attacks for this multi-modal dataset, using conditional, privacy-preserving architectures for generative models, with Bayesian mixtures, for the unbalanced version of the UMNIDS dataset, utilizing unsupervised distinguishability tests and applying parametric tests to measure the difference between the multivariate distribution of the real and synthetic data. Further evaluation is necessary with different differential-privacy architectures and their effect and their affect in privacy scores in combination with fidelity.

V. ACKNOWLEDGMENTS

This work has been supported by the University of Piraeus Research Center (UPRC).

REFERENCES

- [1] A. A. Halimaa, K. Sundarakantham, "Machine Learning Based Intrusion Detection System", IEEE International Conference on Trends in Electronics and Informatics, October, 2019.
- [2] A. A. Sivasamy, B. Sundan, "A Dynamic Intrusion Detection System Based on Multivariate Hotelling's T^2 Statistics Approach for Network Environments", The Scientific World Journal, vol. 2, no. 7, August, 2015.
- [3] A. DasGupta, "Probability for Statistics for Machine Learning: Fundamentals and Advanced Topics", New York, USA, Springer, 2022, pp.1-33.
- [4] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, A. Smola, "A Kernel Two-Sample Test", JMLR, vol. 13, October, 2012, pp.723-773.
- [5] A. Jamalipour, S. Murali, "A Taxonomy of Machine-Learning-Based Intrusion Detection Systems for the Internet of Things: A Survey", IEEE Internet of Things Journal, vol. 9, no. 12, November, 2021, pp.9444-9466.
- [6] A. J. Martineau, K. Fatras, T. Kachman, "Generating and Imputing Tabular Data via Diffusion and Flow-based Gradient-Boosted Trees", Proceedings of Machine Learning Research, vol. 238, May, 2024, pp.1288-1296.
- [7] C. A. Kamhoua, C. D. Kiekintveld, F. Fang, Q. Zhu, "Game Theory and Machine Learning for Cyber Security", IEEE Press Wiley, 2021, pp.47-70.
- [8] D. A. Ammara, J. Ding, K. Tutschku, "Synthetic Network Traffic Generation: A Comparative study", arXiv, October, 2024.
- [9] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A. A. Ghorbani, "CIC-IoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment", Sensors, vol. 23, no. 13, June, 2023.
- [10] G. James, D. Witten, T. Hastie, R. Tibshirani, "An Introduction to Statistical Learning with Applications in R", 2nd ed., New York, USA, Springer, 2021, pp.327-350.
- [11] H. Koubeyssy, A. Amine, M. Kamradt, A. Makhoul, "Survey on Tabular Data Privacy and Synthetic Data Generation in Industry", Applied Intelligence, vol. 55, no. 935, August, 2025.
- [12] I. Ashrapov, "Tabular GANs for uneven distribution", arXiv, 2020.
- [13] I. J. Goodfellow, J. P. Abadie, M. Mirza, B. Xu, D. W. Farley, S. Ozair, A. Courville, Y. Bengio, "Generative Adversarial Networks", NeurIPS, vol. 32, December, 2014, pp. 2672-2680.
- [14] J. Jälkö, E. Lagerspetz, J. Haukka, S. Tarkoma, A. Honkela, S. Kaski, "Privacy-preserving data sharing via probabilistic modeling", Patterns, vol. 2, no. 7, July, 2021.
- [15] J. Jordon, J. Yoon, M. Schaar, "PATE-GAN: Generating synthetic data with differential privacy guarantees", ISLR, May, 2019.
- [16] J. Li, Z. Chen, "Two-sample tests for high-dimensional covariance matrices", The Annals of Statistics, vol. 40, no. 2, April, 2012, pp.911-940.
- [17] J. Yang, L. Wang, "Applying MMD Data Mining to Match Network Traffic for Stepping-Stone Intrusion Detection", Sensors, vol. 21, no. 22, November, 2021.
- [18] K. P. Murphy, "Probabilistic Machine Learning: Advanced Topics", Cambridge, Massachusetts, USA, MIT Press, 2023, pp.55-236.
- [19] L. Chen, Y. Zou, R. D. Cook, "A Regularized Hotelling's T^2 Test for Pathway Analysis in Proteomic Studies", JASA, vol. 106, no. 496, January, 2012, pp.1345-1360.
- [20] L. Hou, Q. Chao, H. Shen, S. Pan, X. Li, X. Cheng, "Conditional GANs with Auxiliary Discriminative Classifier", Proceedings of Machine Learning Research, vol. 162, July, 2022, pp.8888-8902.
- [21] L. Xu, M. Skoularidou, A. C. Infante, K. Veeramachaneni, "Modeling Tabular data using Conditional GAN", NeurIPS, vol. 32, December, 2019.
- [22] M. Alqulaity, P. Yang, "Enhanced Conditional GAN for High-Quality Synthetic Tabular Data Generation in Mobile-Based Cardiovascular Healthcare", Sensors, vol. 24, no. 23, November, 2024.
- [23] M. Arjovsky, S. Chintala, L. Bottou, "Wasserstein Generative Adversarial Networks", Proceedings of Machine Learning Research, vol. 70, January, 2017, pp.214-223.
- [24] M. Cantone, C. Marrocco, A. Bria, "Machine Learning in Network Intrusion Detection: A Cross-Dataset Generalization Study", IEEE Access, vol. 12, October, 2024, pp.50-54.
- [25] M. L. Martin, B. Carro, A. C. Esguevillas, J. Lloret, "Conditional Variational Autoencoder for Prediction and Feature Recovery Applied to Intrusion Detection in IoT", Sensors, vol. 17, no. 9, August, 2017.
- [26] N. Patki, R. Wedge, K. Veeramachaneni, "The Synthetic Data Vault", IEEE International Conference on Data Science and Advanced Analytics, October, 2016, pp.399-410.
- [27] O. Habibi, M. Chemmakha, M. Lazaar, "Imbalanced tabular data modelization using CTGAN and machine learning to improve IoT Botnet attacks detection", Engineering Applications of Artificial Intelligence, vol. 118, February, 2023.
- [28] P. J. Clark, F. C. Evans, "Distance to Nearest Neighbor as a Measure of Spatial Relationships in Populations", Ecology, Ecological Society of America, vol. 35, no. 4, October, 1954, pp.445-453.
- [29] P. Mishra, V. Varadharajan, U. Tupakula, E. S. Pilli, "A Detailed Investigation and Analysis of Using Machine Learning Techniques for Intrusion Detection", IEEE Communications Surveys and Tutorials, vol. 21, no. 1, October, 2019, pp.686-728.
- [30] R. Patgiri, U. Varshney, T. Akutota, R. Kunde, "An Investigation on Intrusion Detection System Using Machine Learning", IEEE Symposium Series on Computational Intelligence, November, 2018, pp.1684-1691.
- [31] S. Bourou, A. Saer, T. H. Velivassaki, A. Voulkidis, T. Zahariadis, "A Review of Tabular Data Synthesis Using GANs on an IDS Dataset", Information, vol. 12, no. 9, September, 2021.
- [32] S. M. Othman, F. M. B. Alwi, N. T. Alsohybe, A. Y. Al-Hashida, "Intrusion detection model using machine learning algorithm on Big Data environment", Journal of Big Data, Springer, vol. 5, September, 2018.
- [33] S. Nowozin, B. Cseke, R. Tomioka, "f-GAN: Training Generative Neural Samplers using Variational Divergence Minimization", NIPS, Barcelona, Spain, June, 2016, pp.1-6.
- [34] S. Wali, Y. A. Farrukh, I. Khan, N. D. Bastian, "Meta: Toward a Unified, Multimodal Dataset, for Network Intrusion Detection Systems", IEEE Data Descriptions, vol. 1, October, 2024, pp.50-54.
- [35] T. M. Cover, J. A. Thoma, "Elements of Information Theory", 2nd ed., Wiley Press, USA, 2006, pp.13-50.
- [36] T. Okadome, "Essentials of Generative AI", Singapore, Springer, 2025, pp.177-194.
- [37] V. Borisov, K. Sessler, T. Leemann, M. Pawelczyk, G. Kasneci, "Language Models are Realistic Tabular Data Generators", ICLR, vol. 70, April, 2023, pp.214-223.
- [38] V. P. Janeja, "Data Analytics for Cybersecurity", Cambridge, United Kingdom, Cambridge University Press, 2011, pp.1-41.
- [39] X. Han, M. Zhou, "Diffusion Boosted Trees", arXiv, June, 2024.
- [40] Q. Zhaozhi, R. Davis, M. Schaar, "Synthcity: a benchmark framework for diverse use cases of tabular synthetic data", NeurIPS, vol. 36, 2023.